

s4.04 — Global video streaming service

Index

- Interview framing
- Requirements — IC3 baseline
- Capacity and consequence — IC3 baseline
- Architecture and flow — IC3 baseline
- API and data model — IC3 baseline
- Upload, transcode, and immutable publish — IC3 baseline
- Adaptive bitrate, CDN, and the DRM boundary — IC3 baseline
- Failure, operations, and security
- Alternatives and IC4 stretch
- Level signal — IC3 baseline vs IC4 stretch
- Interview pushback and validation
- Recall card
- References

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - IC3 baseline is what must land in every phase.
 - IC4 stretch adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: first-hand report attribution 1p3a is retained in the bank.
 - The architecture below is a proposed interview answer.
 - It does not claim to reproduce any streaming provider's implementation or an interviewer's accepted answer.
- OCI framing for a business prompt
 - Control plane: video metadata, processing operations, the published-version pointer, entitlements, and content keys.
 - Data plane: CDN edges, origin shield, object storage serving immutable segments, and the license server.
 - Static stability: a playback session already started must keep playing through a control-plane outage.
 - Blast radius: one region's origin and one CDN point of presence fail alone; a bad manifest affects one video version.
- Why this is hard
 - Naive: upload an MP4 to object storage and hand the player its URL.
 - First break: one bitrate for every network, so mobile viewers rebuffer and seeking downloads bytes the viewer never watches.
 - Second break: a "successful upload" is treated as playable, so half-transcoded renditions and overwritten segments behind an unchanged URL produce broken or inconsistent playback from CDN caches.
 - Third break: long-lived public segment URLs bypass every entitlement check the metadata API performs.
 - Minimum primitives
 - Immutable source versions and immutable rendition artifacts addressed by version.
 - Processing runs keyed by source version and profile version, so retries deduplicate.
 - One published-version pointer flipped in a single conditional transaction.
 - Short-lived scoped playback access to versioned CDN paths.
 - Content encryption with keys held in a key service, delivered only by a license server.
- 🗣️ "Upload is not publish: renditions are immutable artifacts, and one manifest pointer decides what is playable."
- Opening script
 - "I will treat this as a video-on-demand service with an ingest and transcode control plane and a CDN-fronted immutable data plane."
 - "First I confirm catalog size, upload volume, peak concurrent viewers, whether DRM is required, and how fast a takedown must take effect."
 - "Then I show that egress dominates everything, so the CDN hit ratio and immutable versioned URLs are the numbers that decide the design."
 - "The architecture is a resumable upload into object storage, a chunk-parallel transcoder producing versioned renditions, a validation step, and one atomic publish that points playback at a manifest."
 - "The deep dives are the publish state machine across processing and the playback path through adaptive bitrate, CDN caching, signed access, and the DRM boundary."
 - "I will close with failure containment, tenant isolation, and where live streaming or per-title encoding changes the design."

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Scope, clarifying answers, primary invariant
Estimates and contract	11:00–17:00	Storage, transcode CPU, egress; API semantics
Architecture	17:00–30:00	Diagram, plane split, one publish trace and one playback trace
Deep dive	30:00–45:00	Processing runs and atomic publish; ABR, CDN keys, DRM boundary
Operations and tradeoffs	45:00–50:00	Failure table, degraded playback, tradeoff, wrap script

Phase	Clock	Deliverable
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Video on demand or live?	On demand; live is a stretch	Immutable artifacts and one publish pointer; live needs sliding manifests
Scale?	10M videos, 100k uploads/day, 10M peak concurrent viewers	Egress dominates; storage is petabytes; transcode is a compute fleet
Formats and devices?	HLS and DASH over CMAF segments; web, mobile, TV	One packaging step serves both manifests; device capabilities select codecs
DRM required?	Yes for licensed content; signed URLs only for creator uploads	Adds content keys, a license server, and encrypted packaging
Who is the tenant?	A content owner or studio in one deployment	Tenant on every key, storage prefix, and content key; quotas on processing minutes
Takedown deadline?	Discoverability within 1 min; playback within 15 min	Bounds the signed access TTL and the CDN behavior on revocation
What survives a control-plane outage?	Started sessions keep playing; new starts may fail	Immutable CDN paths and issued tokens do not depend on the metadata store

- Functional scope
 - Accept large uploads with resume and integrity checks.
 - Transcode a source version into a ladder of immutable renditions and package them for HLS and DASH.
 - Validate and publish exactly one playable version per video.
 - Start playback with an entitlement check and short-lived scoped access.
 - Serve segments through a CDN with adaptive bitrate selection on the client.
 - Encrypt licensed content and deliver keys through a license server.
 - Remove or replace a video without breaking cached content for other videos.

Requirement	Target	Enforced by
Playback start time	p95 < 2 s to first frame	Small master manifest, 4 s segments, first segments cached at the edge
Rebuffer ratio	< 0.5% of watch time	ABR ladder from 400 kbps to 6 Mbps; client buffer target 30 s
CDN offload	≥ 95% edge hit ratio; origin egress ≤ 5%	Immutable versioned URLs with one-year cache lifetime; origin shield per region
Time to ready	p95 < 30 min for a 60 min source	Chunk-parallel transcoding; renditions in parallel
Playback availability	99.99% for segment delivery	Multi-PoP CDN; object storage origin in two regions
Control-plane availability	99.9% for upload, processing, and playback-session APIs	Stateless API fleet; replicated metadata store
Publish atomicity	1 published version per video at any instant	Conditional update of published_version with expected row version
Signed access TTL	15 min per session, renewable while entitled	Token bound to tenant, video, version, and session
Takedown	Discoverability 1 min; playback within 15 min	Publish pointer cleared; token renewal refused; CDN purge for the manifest path
Source durability	Provider-documented object storage durability; retain source indefinitely by default	Object storage with versioning; deletion only through the retention policy

- Invariants
 - A rendition segment, once written under a versioned path, is never overwritten.
 - A processing run is identified by (video_id, media_version, profile_version) and produces the same artifacts on retry.
 - published_version points only at a version whose required renditions are all READY and validated.
 - A manifest references artifacts of exactly one media version.
 - A playback token grants access to one tenant, video, version, and session for a bounded time.
 - Content keys never appear in a manifest or a log; the license server is the only key path to a device.
- Non-goals
 - Live and low-latency streaming in the first release.
 - Recommendation and search; that is s4.06 territory.
 - Building a CDN; the design consumes a commercial or platform CDN.
 - Implementing DRM client security; the device's content decryption module belongs to the DRM vendor.
 - Ad insertion.

Capacity and consequence — IC3 baseline

- Storage
 - 10M videos with an average length of 20 min and a source at about 5 GB/h is about 1.7 GB per source, roughly 17 PB of sources.
 - A six-rung H.264 ladder at 0.4, 0.8, 1.5, 3, 4.5, 6 Mbps sums to about 16 Mbps, about 2 MB/s, so 20 min of renditions is about 2.4 GB.
 - 10M × 2.4 GB is about 24 PB of renditions; a second codec for the top of the catalog adds a fraction of that.
 - Decision: tier storage by popularity and age.
 - Sources and cold renditions move to infrequent-access tiers after 90 d without views.
 - Encode a second codec such as AV1 only for the top 5% of videos by views.
- Object count
 - 4 s segments make a 20 min video 300 segments per rendition, about 1,800 segment objects plus playlists.
 - 10M videos is on the order of 18 billion objects.
 - Decision: the object key is `media/{tenant}/{video}/{mediaVersion}/{rendition}/{segment}`.
 - Object storage owns the namespace; the metadata store holds versions and renditions, never per-segment rows.
- Transcoding
 - 100k uploads/day of 20 min each, at an assumed 1× real time per rendition on one core, is about 2 core-hours per video, about 200k core-hours/day.
 - That is about 8.3k cores busy around the clock, more at the daily upload peak.
 - Decision: split the source into 2–4 min chunks and transcode chunks and renditions in parallel.
 - Wall time for a 60 min source drops from hours to under 30 min.
 - The worker pool is queue-fed and autoscaled; priority by tenant tier, not first come.
- Egress
 - 10M concurrent viewers at an assumed average of 3 Mbps is 30 Tbps.
 - At a 95% edge hit ratio the origin still sees 1.5 Tbps; at 99% with a shield tier it sees 300 Gbps.
 - Decision: a two-tier CDN with an origin shield per region and immutable URLs is a requirement, not an optimization.
 - Origin capacity is sized for the shield miss rate, not for viewer count.
- Playback sessions
 - 10M concurrent viewers with an average 20 min watch is about 8.3k session starts per second at peak.
 - Each start is an entitlement check, a token signature, and one manifest URL.
 - Decision: the playback API is stateless with cached entitlements and asymmetric token signing.
 - The metadata store sees one read per start, not one per segment.
- Manifests
 - A master manifest is a few kilobytes; a rendition playlist for 300 segments is on the order of 20 KB.
 - Decision: manifests are immutable per version and cached at the edge like segments.
 - Only the metadata store's `published_version` pointer changes, and the client never fetches it from the CDN.

Architecture and flow — IC3 baseline

```
[control plane: ingest and publish]
Creator -> upload API (resumable parts, s1.09) -> object storage: source/{tenant}/{video}/{mediaVersion}
      |
      +--> processing orchestrator (202 + operation id) -> queue -> transcode workers (chunk-parallel)
                                   -> packager (CMAF segments, HLS+DASH playlists,
                                   CENC encryption with content key from key service)
                                   -> validator (playlists resolve, durations, hashes)
      +--> metadata store (videos, video_versions, video_renditions, operations, outbox)
          publish = conditional flip of published_version -> outbox -> search, notifications

[data plane: playback]
Viewer -> playback API (entitlement, device caps) -> signed token + versioned manifest URL
Viewer -> CDN edge -> origin shield (per region) -> object storage origin (versioned immutable paths)
Viewer -> license server (entitlement + key service) -> device CDM decrypts segments
```

- Upload API owns resumable ingest of the source.
 - Mechanism bought: fixed parts with checksums and a frozen manifest, reused from [s1.09](#).
 - Cost paid: a completed upload only creates a source version; nothing is playable until processing and publish finish.
- Processing orchestrator owns the run state machine.
 - Mechanism bought: a work request per (video, media_version, profile_version) with an operation id, so retries and duplicate submissions collapse to one run.
 - Cost paid: every creator sees an asynchronous 202 and must poll or subscribe for readiness.
- Transcode workers and packager own artifact production.
 - Mechanism bought: chunk-level parallelism bounds wall time; CMAF segments feed both HLS and DASH from one set of bytes.
 - Cost paid: chunk boundaries must align across renditions so ABR switches land on keyframes, which constrains the encoder settings.
- Validator owns the readiness decision.
 - Mechanism bought: it checks that every playlist reference resolves, durations sum to the source, and artifact hashes match before a version is `READY`.

- Cost paid: one more pass over the artifacts, on the order of minutes per version.
- Metadata store owns videos, versions, renditions, and the publish pointer.
 - Mechanism bought: one relational store per region gives a transactional flip with an outbox row.
 - Cost paid: single-region writes for the control plane; a store outage stops new publishes and new playback starts.
- CDN and origin shield own delivery.
 - Mechanism bought: immutable versioned paths mean a cache never needs invalidation for segments; the shield collapses edge misses per region.
 - Cost paid: an invalid artifact that reached the CDN is fixed by publishing a new version, never by rewriting the path.
- Playback API and license server own authorization.
 - Mechanism bought: a short-lived token scoped to one version and session; keys leave the key service only wrapped for a device's content decryption module.
 - Cost paid: revocation is bounded by the token TTL unless the CDN validates tokens online.

→ Creator calls `POST /videos`, then `POST /videos/{id}/uploads` and completes a resumable upload into the source path for `media_version = n`.

→ Creator or a hook calls `POST /videos/{id}/processing-runs` with the source object version and profile set; the API returns `202` with an operation id.

→ Workers transcode chunks per rendition in parallel, the packager writes CMAF segments and playlists under the versioned path, and each rendition row moves `PENDING` → `READY`.

→ The validator checks the playlists and hashes and marks `video_versions.state = READY`.

→ The publish step updates `videos.published_version = n` with `WHERE version = expected`, inserts an outbox row, and commits.

→ A viewer calls `POST /videos/{id}/playback-sessions`; the API checks entitlement, reads `published_version`, and returns a signed URL for that version's master manifest.

→ The player fetches the manifest and segments from the CDN edge; misses go through the origin shield to object storage.

→ For encrypted content the player asks the license server for the key by key id; the server checks entitlement and returns the key wrapped for the device.

API and data model — IC3 baseline

- Paths omit the `/v1` prefix and follow the [shared API conventions](#).
 - **I** means Idempotency-Key is required.
 - **V** means a strong If-Match validator is required.

Surface	Contract	Idempotency	Conflict/error behavior
<code>POST /videos</code>	<code>{title, visibility}</code> ; returns <code>201 {videoId}</code>	I	<code>409</code> on key reuse with a different payload
<code>POST /videos/{id}/uploads</code>	<code>{sizeBytes, sha256}</code> ; returns an upload session per s1.09	I	<code>409</code> when an upload for the same declared hash is already complete
<code>POST /videos/{id}/processing-runs</code>	<code>{sourceObjectVersion, profileSet}</code> ; returns <code>202 {operationId}</code>	I; run identity is <code>(video, media_version, profile_version)</code>	Same identity returns the existing operation; <code>409</code> if the source version does not exist
<code>GET /operations/{id}</code>	<code>ACCEPTED\ RUNNING\ SUCCEEDED\ FAILED</code> , per-rendition progress	Read	Authoritative answer after a lost <code>202</code>
<code>POST /videos/{id}/publish</code>	<code>{mediaVersion, expectedVersion}</code> ; returns <code>200 video</code>	I	<code>409</code> if the version is not <code>READY</code> or the expected row version is stale
<code>GET /videos/{id}</code>	<code>{state, metadata, availableRenditions}</code>	Read	Only the published version appears playable; <code>404</code> across tenants
<code>POST /videos/{id}/playback-sessions</code>	<code>{deviceCapabilities}</code> ; returns <code>201 {manifestUrl, expiresAt, sessionId}</code>	I per viewer	<code>403</code> when not entitled; <code>409</code> when no version is published
<code>POST /playback-sessions/{id}/renewals</code>	Returns a fresh token for the same version	Idempotent by session	<code>403</code> once entitlement is revoked
CDN <code>GET /media/{tenant}/{video}/{version}/manifest.m3u8</code> or <code>.mpd</code>	Signed token in query or header	Read	<code>403</code> on an expired or mismatched token

Surface	Contract	Idempotency	Conflict/error behavior
CDN GET /media/{tenant}/{video}/{version}/{rendition}/{segment}	Signed token; range requests where supported	Read	403 on token mismatch; a token only ever names a version that was published when it was issued, so the edge performs no publish check per request
POST /drm/licenses	{sessionId, keyId, deviceChallenge}; returns a license for the device	Idempotent per session and key	403 when the session is not entitled to the key's version
DELETE /videos/{id}	Returns 202 removal operation	V	Clears published_version, refuses renewals, purges manifest paths, schedules blob cleanup

State	Key	Fields/invariant
Video	(tenant_id, video_id)	Owner, title, visibility, state, published_version, row version
Media version	(tenant_id, video_id, media_version)	Source ref and hash, profile version, state PENDING\ READY\ FAILED\ RETIRED, manifest ref
Rendition	(tenant_id, video_id, media_version, rendition_id)	Codec, bitrate, resolution, playlist ref, state; immutable once READY
Processing run	(tenant_id, video_id, media_version, profile_version)	Operation id, chunk progress, attempt fence
Content key	(tenant_id, key_id)	Wrapped key in the key service, video and media version, rotation generation; never plaintext at rest here
Playback session	(tenant_id, session_id)	Viewer, video, media version, device class, issued and expiry times, revoked flag
Idempotency record	(tenant_id, caller_id, operation, intent_key)	Request hash and stored response
Outbox	(tenant_id, event_id)	Aggregate, version, payload; same transaction as the state change

- Reference DDL (from the companion; content_keys is the DRM addition)

```

CREATE TABLE videos (
  tenant_id text NOT NULL, video_id text NOT NULL, owner_id text NOT NULL,
  title text NOT NULL, visibility text NOT NULL, state text NOT NULL,
  published_version bigint NOT NULL, created_at timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, video_id)
);
CREATE INDEX videos_owner ON videos (tenant_id, owner_id, created_at DESC, video_id);
CREATE TABLE video_versions (
  tenant_id text NOT NULL, video_id text NOT NULL, media_version bigint NOT NULL,
  source_ref text NOT NULL, source_hash text NOT NULL, profile_version text NOT NULL,
  state text NOT NULL, manifest_ref text,
  PRIMARY KEY (tenant_id, video_id, media_version),
  FOREIGN KEY (tenant_id, video_id) REFERENCES videos
);
CREATE TABLE video_renditions (
  tenant_id text NOT NULL, video_id text NOT NULL, media_version bigint NOT NULL,
  rendition_id text NOT NULL, codec text NOT NULL, bitrate_bps bigint NOT NULL,
  width integer NOT NULL, height integer NOT NULL, playlist_ref text NOT NULL,
  state text NOT NULL,
  PRIMARY KEY (tenant_id, video_id, media_version, rendition_id),
  FOREIGN KEY (tenant_id, video_id, media_version) REFERENCES video_versions
);
CREATE TABLE content_keys (
  tenant_id text NOT NULL, key_id text NOT NULL,
  video_id text NOT NULL, media_version bigint NOT NULL,
  wrapped_key_ref text NOT NULL,          -- handle in the key service; never key bytes
  generation integer NOT NULL, state text NOT NULL,
  PRIMARY KEY (tenant_id, key_id),
  FOREIGN KEY (tenant_id, video_id, media_version) REFERENCES video_versions
);

```

- Conventions
 - tenant_id and the caller come from authenticated context, never from the body.
 - expectedVersion on publish is a domain precondition on the video row, so its failure is 409.
 - Blob layout is media/{tenant}/{video}/{mediaVersion}/{rendition}/{segment}; the manifest records segment order, durations, and hashes.

- The playback token is a signed claim set {tenant, video, version, session, exp}; the CDN verifies the signature with a public key and never calls the metadata store per segment.

Upload, transcode, and immutable publish — IC3 baseline

```
media_version:  PENDING --all required renditions READY and validated--> READY --publish--> (published) --newer version published-->
RETIRED
rendition:      PENDING --any required rendition FAILED after retries--> FAILED
                PENDING --chunks encoded, packaged, hashed--> READY
                PENDING --encoder error or hash mismatch--> FAILED
publish:        UPDATE videos SET published_version = n, version = version + 1
                WHERE tenant_id = t AND video_id = v AND version = expected
                AND EXISTS (SELECT 1 FROM video_versions WHERE ... AND media_version = n AND state = 'READY')
```

- Processing run identity
 - The run key is (video_id, media_version, profile_version).
 - A retry, a duplicate hook, or a second operator click finds the existing run and returns its operation id.
 - A new profile version, for example a new ladder or codec, is a new run against the same source and writes under a new media_version.
 - Each rendition and each chunk carries an attempt fence so a worker that lost its lease cannot mark progress after a takeover.
- Chunk-parallel transcoding
 - Split the source at 2–4 min boundaries and force a keyframe at each boundary in every rendition.
 - Encode chunks across workers, then package into 4 s CMAF segments aligned across the ladder.
 - Aligned segments are what make an ABR switch seamless; misaligned renditions produce a stall or a decode error at the switch.
 - A chunk failure retries the chunk, not the video; a rendition fails only after bounded attempts.
- Validation before READY
 - Every playlist reference resolves to an object with the recorded hash.
 - Segment durations sum to the source duration within tolerance.
 - Each rendition decodes its first and last segment.
 - A missing optional rendition does not block; a missing required rendition does.
 - "Partial renditions do not make the whole video ready" is the rule to say aloud.
- Publish
 - The publish step is one conditional UPDATE with the expected row version and a READY check, plus an outbox row.
 - Two publishes racing for the same video serialize on the row version; the loser gets 409 and re-reads.
 - The previous version becomes RETIRED but its artifacts remain until no session can reference them.
 - Rollback is publishing the previous READY version again; it needs no re-encode because nothing was overwritten.
- Immutability and CDN correctness
 - No process ever writes twice to the same {version}/{rendition}/{segment} path.
 - A fix is always a new media_version, so a CDN cache holding old bytes is still consistent with its own manifest.
 - Rewriting segments behind an unchanged manifest URL is the failure this rule prevents; different edges would hold different bytes for the same path.
- Garbage collection
 - A retired version is deleted only when it has no live manifest reference, no unexpired playback session, and no retention hold.
 - The deletion is an async operation with its own idempotent record; a lost response is safe to retry.
- Consistency boundary
 - The metadata store is authoritative for what is published.
 - Object storage and the CDN are authoritative for bytes, and bytes never change under a path.
 - The two are reconciled by immutability, not by invalidation.
- Exact failure
 - A worker crashes after writing half of a rendition's segments.
 - Its lease expires, a new worker takes over, and it re-encodes only chunks whose segments are missing or fail their hash.
 - The rendition stays PENDING, the version stays unpublished, and viewers keep the previous published version.

Adaptive bitrate, CDN, and the DRM boundary — IC3 baseline

- Adaptive bitrate on the client
 - The master manifest lists renditions with bitrate, resolution, and codec; the player picks based on its measured throughput and buffer level.
 - Segments of 4 s give a switch granularity of 4 s; shorter segments switch faster but multiply requests and playlist size.
 - The ladder 0.4, 0.8, 1.5, 3, 4.5, 6 Mbps covers 240p to 1080p; a 4K rung at 15 Mbps is added per device class.
 - A 30 s buffer target absorbs throughput dips; startup uses a low rung and one or two segments to reach first frame quickly.
 - The server does not choose the bitrate; it only guarantees aligned, honestly labeled renditions.
- Manifest and segment addressing
 - HLS uses a master playlist pointing at rendition playlists; DASH uses one MPD describing adaptation sets.
 - Both reference the same CMAF segments under the versioned path.
 - The playback API hands out the manifest URL for the published version, so the client never asks the CDN "which version is current".
 - Byte-range addressing into one file per rendition is an alternative to discrete segments; it reduces object count but complicates partial caching and hashing.

- CDN caching
 - Cache key is the versioned path; the token is validated at the edge and excluded from the cache key.
 - `Cache-Control: public, max-age=31536000, immutable` on segments and manifests.
 - Origin shield per region collapses concurrent misses for a newly popular segment into one origin fetch.
 - A 95% edge hit ratio is the floor; the shield tier pushes the origin miss rate toward 1%.
 - Popular launches are prewarmed by fetching the first minute of every rendition into shields ahead of release.
- Signed access
 - The playback API signs a token `{tenant, video, version, session, exp}` with a private key; edges verify with the public key.
 - TTL is 15 min; the player renews through `POST /playback-sessions/{id}/renewals` while entitled.
 - Revoking entitlement refuses renewal, so playback stops within the TTL; that is the documented takedown bound.
 - Binding the token to a client IP is a knob that hurts mobile users on changing networks; session binding is the default.
- DRM boundary
 - The packager encrypts segments with a per-version content key using common encryption, so one set of bytes serves several DRM systems.
 - The manifest carries the key id and the license server URL, never the key.
 - The license server authenticates the session, checks entitlement for the version, fetches the key from the key service, and returns it wrapped for the device's content decryption module.
 - The key service holds keys wrapped by a master key in a KMS or HSM; this service stores a `wrapped_key_ref`, not bytes.
 - What this design owns: key generation, key custody, license policy, and the entitlement check.
 - What the DRM vendor owns: the device decryption module, output protection, and device attestation.
 - Persistent licenses with a bounded duration let an already-started session survive a license-server outage; new sessions cannot.
- Static stability
 - An issued token and an immutable CDN path do not depend on the metadata store, so a started session keeps playing through a control-plane outage until the token expires.
 - Extend the renewal grace to the outage budget so a 30 min control-plane outage does not stop viewers who were entitled when it began.
 - New session starts fail with 503 rather than serving an unverified entitlement.
- Exact failure
 - The license server is unavailable.
 - Clear and signed-URL content keeps starting; encrypted content starts fail and sessions with a persistent license keep playing.
 - The alarm is license failure rate; the mitigation is a second license-server region and bounded persistent licenses.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Transcode worker crash mid-rendition	One version of one video	Lease expiry; oldest <code>PENDING</code> rendition age	Version stays unpublished; previous version keeps serving	Takeover re-encodes missing chunks under a new fence
Rendition fails validation	One version	Hash or duration mismatch	Version <code>FAILED</code> ; publish refused	Re-run with a new <code>media_version</code> after fixing the profile or source
Two publishes race	One video	409 on expected version	One wins; the loser re-reads	Republish if still desired
Segment written twice under one path	One version's CDN consistency	Write guard on existing key; hash audit	Path is refused; version stays <code>PENDING</code>	Publish a new version; never purge and rewrite
CDN point of presence down	Viewers routed to that PoP	Edge health; error rate by PoP	DNS or anycast shifts viewers to a neighbor PoP	Automatic once the PoP recovers
Origin shield down in one region	Miss traffic for that region	Origin request spike	Edges fall back to a second shield or direct origin with higher latency	Restore shield; prewarm hot content
Object storage region outage	Origin for that region	Origin 5xx rate	Shields fetch from the second-region copy of the versioned path	Cross-region replication catches up; no path changes
Metadata store unavailable	New publishes and new playback starts in one region	Health check; error rate	Started sessions keep playing on issued tokens and immutable paths; renewals use the grace window; new starts get 503	Fail over the replica; outbox replays
License server down	Encrypted content starts	License failure rate	Persistent licenses keep playing; new encrypted starts fail	Second region; bounded persistent licenses
Entitlement revoked mid-session	One viewer	Renewal refused	Playback stops at token expiry, within 15 min	None needed; bound is documented
Upload session lost	One creator's upload	Upload session state	Resume per s1.09	List verified parts and continue

- Observe
 - Time to ready per version and per rendition, and failed-rendition ratio.
 - Edge hit ratio, shield hit ratio, and origin egress by region.
 - Startup time, rebuffer ratio, and bitrate distribution by device class and region.
 - Token rejection rate at the edge and license failure rate.

- Oldest `PENDING` version and outbox lag.
- Retired versions awaiting garbage collection and their storage bytes.
- Retry discipline
 - Chunk encodes retry with bounded attempts; a rendition fails after the bound instead of retrying forever.
 - Processing-run submissions retry with the same run identity, never a new one.
 - Players retry a segment fetch with backoff and switch down a rung before giving up; a `403` on a token triggers renewal, not a retry loop.
 - License requests retry a bounded number of times; a `403` is a corrective action, not a retry.
- Tenant isolation
 - Every key and every storage prefix starts with `tenant_id`; the CDN path carries the tenant so a token for one tenant cannot match another's path.
 - Content keys are per tenant and per version; a key never encrypts content of two tenants.
 - Quotas cap processing minutes per day and storage bytes per tenant; a burst from one studio queues behind its own quota, not everyone's.
 - A caller receives `404`, not `403`, for another tenant's video IDs.
- Rollout and rollback safety
 - A new transcoding profile is a new `profile_version`; existing published versions are untouched, and backfill runs by popularity under its own quota.
 - Manifest format changes are canaried by device class; a player that cannot parse a new manifest keeps a compatible fallback listed first.
 - Rollback of a bad publish is a flip of `published_version` to the previous `READY` version, with no re-encode and no CDN purge.
 - CDN configuration changes, such as token verification rules, roll out per PoP with a synthetic playback probe before promotion.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Progressive download	Serve one MP4 with range requests	Simplest pipeline	One bitrate; seeking wastes bytes; no DRM segmentation	Small catalogs and reliable networks
HLS and DASH over CMAF	One segment set, two manifest formats	Every device class from one artifact set	Two manifest generators to keep consistent	Baseline
Discrete segments	One object per segment	Simple caching and hashing	Billions of objects	Baseline with an object store
Byte-range single file	One file per rendition with a byte-range index	Fewer objects	Partial caching and integrity checks are harder	Object count is the binding limit
Pre-packaging	Package once at processing time	Cheap playback path; fully cacheable	Storage for every format	Baseline
Just-in-time packaging	Package from a mezzanine at request time	Less storage; new formats without re-encode	Compute on the playback path; cache misses cost CPU	Long-tail catalogs where storage dominates
Fixed ladder	Same rungs for every title	Simple, predictable	Wastes bits on simple content	Baseline
Per-title encoding	Ladder chosen from content complexity	20–40% fewer bits at equal quality (order of magnitude, provider-reported)	Analysis pass per title	Egress cost dominates
Signed URLs only	Token-gated clear content	No license server	Bytes are reusable once fetched	Creator content without licensing terms
DRM with common encryption	Encrypted segments plus a license server	Licensed content compliance	Vendor modules, key custody, license availability	Contract requires it

- IC4 stretch — live streaming
 - The manifest becomes a sliding window that changes every segment, so immutability moves from the manifest to the segments.
 - Segment duration drops to 1–2 s and the encoder runs in real time with redundancy per region.
 - The publish pointer is replaced by a stream state machine, and CDN caching of the playlist uses a short TTL.
- IC4 stretch — 10× load
 - 100M concurrent viewers is 300 Tbps; multi-CDN with a steering service that shifts traffic by measured quality per region.
 - Per-title encoding and a second codec cut bits per viewer, which is the only lever that scales with egress.
 - Playback session starts at 83k/s need entitlement caching per viewer with a short TTL and regional playback APIs.
- IC4 stretch — cost and blast radius
 - Egress is the dominant cost; storage tiering and codec efficiency are second.
 - Cell the origin and metadata store per region so one region's outage strands one region's publishes and starts.
 - Degrade by capping the top rung per region before refusing starts.
- IC4 stretch — migration and rollout
 - A codec migration is a backfill ordered by views under a separate quota, with the old profile version still published until the new one validates.
 - Per-video rollback stays a pointer flip throughout the migration.
- Weakest assumption
 - A 95% edge hit ratio.
 - A long-tail catalog or a sudden regional launch can drop the hit ratio, and origin egress rises tenfold with it.
 - Mitigation: an origin shield per region, prewarming for launches, and an origin capacity alarm keyed to shield miss rate.

- Over-engineering warning
 - Do not design live or low-latency streaming into a video-on-demand prompt.
 - Do not start with per-title encoding or just-in-time packaging; measure egress and storage first.
 - Do not build a CDN; consume one and own the cache-key and token contract.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Catalog and viewer scale, DRM requirement, takedown bound, upload-is-not-publish invariant	Names live streaming and per-title encoding as non-goals and what cutting them buys	Negotiates scope instead of accepting it
Estimates	24 PB renditions, 200k core-hours/day, 30 Tbps egress	Lets egress force the two-tier CDN and lets object count force the key layout	A number changes a decision
Architecture	Ingest and publish control plane versus CDN data plane with one publish trace and one playback trace	States the cost of each split: async readiness and revocation bounded by token TTL	Every component has a mechanism and a price
Deep dive	Run identity, chunk fences, validation gate, conditional publish, immutable paths	Segment alignment for ABR, shield tiers, token scope, DRM key custody boundary	Reasons across the metadata store, object storage, and CDN boundary
Operations	Failure table, degraded playback, static stability of started sessions	Per-region cells, tenant quotas, profile-version rollout and pointer rollback	Volunteers degradation and rollout before being asked
Wrap	Recap and tradeoff	Names the hit-ratio assumption first	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: The upload succeeded, so why is the video not playable yet?
 - **IC3 baseline**
 - Upload creates a source version; playable means every required rendition is `READY`, validated, and the publish pointer flipped.
 - **IC4 stretch**
 - Expose per-rendition progress on the operation so creators see readiness as a percentage, and offer a low-rung early publish only if the product accepts a two-step publish.
- Q2: A transcode job runs twice.
 - **IC3 baseline**
 - The run identity (`video`, `media_version`, `profile_version`) deduplicates the submission; the second call returns the first operation.
 - **IC4 stretch**
 - Chunk fences stop a stale worker from marking progress after takeover; artifacts are deterministic per path so a duplicate write is refused, not overwritten.
- Q3: How do you fix a bad segment that is already cached on the CDN?
 - **IC3 baseline**
 - Publish a new `media_version`; never rewrite under the old path.
 - **IC4 stretch**
 - Purge only the manifest path of the bad version if a takedown is needed; segments under a versioned path can stay cached because nothing references them.
- Q4: Entitlement is revoked while the viewer is watching.
 - **IC3 baseline**
 - Renewal is refused, so playback stops within the `15 min` token TTL, and the bound is documented.
 - **IC4 stretch**
 - Shorten the TTL for high-risk content or validate tokens online at the edge, and say what that costs in edge latency and control-plane dependency.
- Q5: The metadata store is down for 30 minutes.
 - **IC3 baseline**
 - Started sessions keep playing on issued tokens and immutable paths; new starts and publishes fail with `503`.
 - **IC4 stretch**
 - A renewal grace window equal to the outage budget keeps entitled viewers watching; the license server caches entitlements for the same window.
- Q6: Where are the DRM keys?
 - **IC3 baseline**
 - In a key service wrapped by a KMS or HSM master key; this design stores a reference, and the license server is the only path to a device.
 - **IC4 stretch**
 - Rotate the content key per media version and audit every license issuance; the vendor's decryption module owns device security, and saying that boundary aloud is the signal.
- Q7 (premise reversal): Why transcode ahead of time at all instead of packaging just in time from one mezzanine?
 - **IC3 baseline**
 - Just-in-time packaging puts CPU on the playback path and makes every cache miss expensive, which is the wrong tradeoff when egress dominates and the catalog is watched.
 - The publish invariant is also harder to state when artifacts are created on demand.

- **IC4 stretch**
 - For a long-tail catalog where storage dominates, package just in time behind the shield from a single mezzanine, and keep pre-packaged artifacts only for the head of the catalog.
- Q8: Now make it live.
 - **IC3 baseline**
 - The manifest is no longer immutable; segments still are, and the playlist gets a short cache TTL.
 - **IC4 stretch**
 - Real-time redundant encoders per region and a stream state machine replace the publish pointer, and latency targets drive segment duration down to 1–2 s.
- Validation scenarios
 - Worker crash with half a rendition written, then takeover.
 - Duplicate processing-run submission with the same source and profile.
 - Publish attempted while one required rendition is `PENDING`.
 - Two publishes racing on one video.
 - Segment write attempted to an existing versioned path.
 - Metadata store down during playback, then a renewal inside the grace window.
 - License server down for encrypted content with and without persistent licenses.
 - Tenant A token presented for tenant B's CDN path.
 - Origin shield loss in one region during a launch.

Recall card

- “Upload is not publish; the publish pointer is the only thing that makes a version playable.”
- “Artifacts are immutable under versioned paths, so the CDN is corrected by publishing, never by purging.”
- “Egress dominates: the hit ratio and the shield tier are the design, the origin is sized for misses.”
- “A token grants one version for one session for 15 min; revocation is bounded by that TTL.”
- “Keys live in the key service and reach a device only through the license server, wrapped for its decryption module.”
- Wrap-up script
 - “The design is a resumable upload into object storage, chunk-parallel transcoding into immutable versioned renditions, a validation gate, and one conditional publish that points playback at a manifest served through a two-tier CDN with signed access and a DRM license boundary.”
 - “The key invariant is that a published version references only `READY` validated artifacts and no path is ever written twice, which is what keeps CDN caches consistent without invalidation.”
 - “The main tradeoff is pre-packaging everything: it costs petabytes of storage and asynchronous readiness, in exchange for a playback path that is fully cacheable and independent of the control plane.”
 - “My weakest assumption is the 95% edge hit ratio; a long-tail catalog or a regional launch can break it, so I would validate shield miss rates and prewarming first.”

References

- [Ranked question and report provenance](#).
- [OCI role framing](#).
- [Detailed API and schema companion](#).
- [Shared API conventions](#): `I` and `V` semantics, 202 operations, error shapes.
- Reference check: 2026-09-10.
 - Official documentation supports the mechanisms cited next to each link.
 - Catalog size, ladder bitrates, segment duration, token TTL, hit ratio, and transcode cost remain proposed interview assumptions.
- [RFC 8216, HTTP Live Streaming](#): master and media playlists, segment references, key delivery attributes.
- [Apple HLS authoring specification](#): segment duration guidance and ladder practice.
- [DASH Industry Forum](#): MPD structure, adaptation sets, and common encryption guidelines.
- [OCI Object Storage overview](#): durability, versioning, and tiering used for sources and renditions.
- [OCI pre-authenticated requests](#): time-bounded object access as the platform analogue of a signed URL.
- [OCI Vault key overview](#): HSM-backed master keys that wrap content keys.
- [Netflix per-title encode optimization](#): the content-adaptive ladder cited as the IC4 alternative.
- [Transactional outbox](#): publish events in the same transaction as the pointer flip.
- Sibling designs
 - [s1.09 fault-tolerant file upload](#): the source upload protocol.
 - [s3.01 async control-plane resource API](#): the work-request contract for processing runs.
 - [s2.04 HSM key management](#): key custody pattern reused for content keys.
 - [s5.02 distributed object storage](#): origin storage background.